Linux Embarqué

But :
Executer en mode user un code écrit en C sur linux embarqué sur une carte Zedboard basé sur un processeur ARM Cortex A9.

1-Materiels :

-Carte SD
-Zedboard
-Pc équipé de Linux, vivado/vitis (Xilinx) qui est le poste de travail.

Zedboard



carte SD

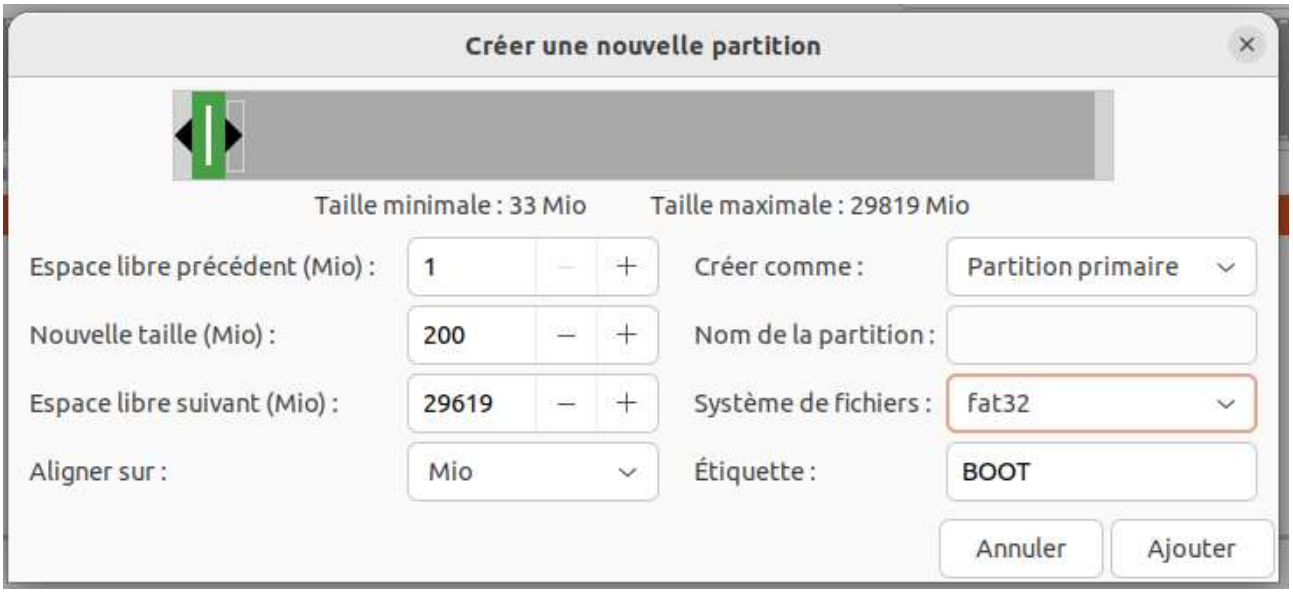


2-Configuration

Il faut configurer principalement la carte SD, car c'est elle qui va contenant les fichiers de lancement et d'exécution de Linux et aussi le fichier exécutable obtenu après un cross-compilation. Pour cela on suit les étapes suivantes

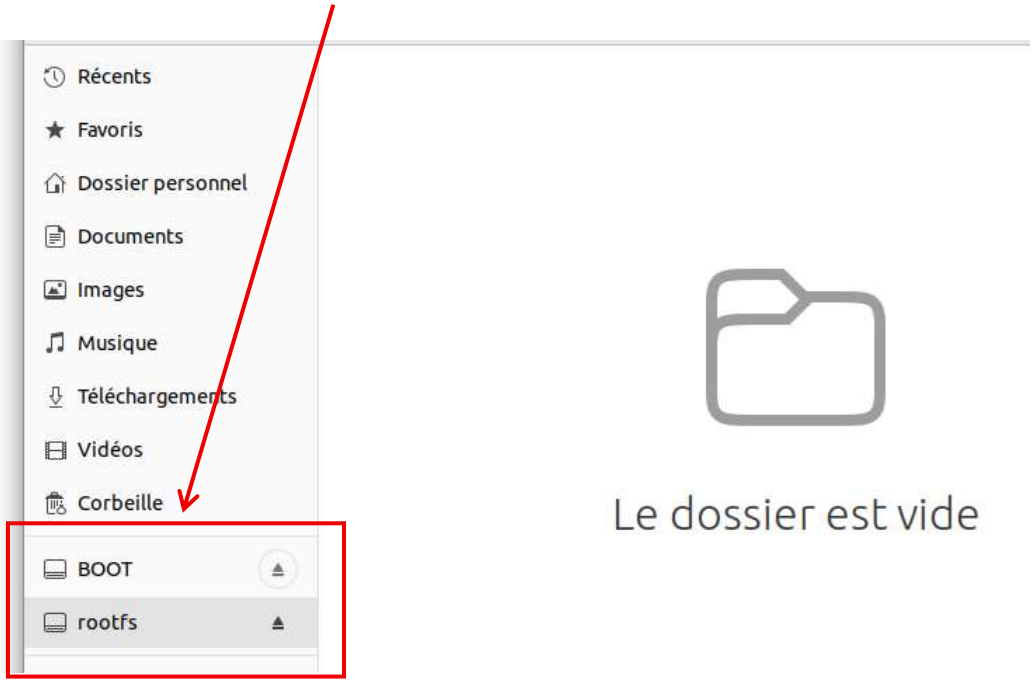
a-formater la clé au format adéquat EXT3 :

On tape dans l'invite de commande « sudo GParted », mais il faut que GParted soit déjà installé. Mais même si ce n'est pas le cas on le proposera dans l'invite de commande.
La carte SD doit contenir 2 partitions, une appelé BOOT qui renferme les fichiers de boot et l'autre rootfs qui renferme l'image Linux qui permet d'utiliser linux,



On fait de même pour la partition rootfs, en utilisant toute la mémoire restante et en la mettant au format EXT3.

On a le résultat suivant :



b-copier le fichier rootfs dans la partition rootfs

On télécharge le fichier rootfs.tar.gz sur internet. Il faut le décompresser dans la partition rootfs. On peut aussi le décompresser sur l'ordinateur et le copier dans la partition rootfs. Dans les 2 cas il va falloir monter la partition.

-sudo mount /dev/sda2 /mnt :
Cette commande **s**ignifie "Monte la partition /dev/sda2 dans le dossier /mnt pour pouvoir accéder à ses fichiers." Avant ça il va falloir créer un dossier mnt dans lequel on va mettre notre fichier décompressé

-Sda2 : signifie 2e partition. BOOT est la 1ere (sda1)

L'opération de montage :
Cette opération relie en quelque sorte le contenu physique de cette partition au répertoire /mnt de ton système. Avant le montage, /mnt est un simple dossier vide sur ton ordinateur. Après le montage, ce dossier reflète exactement le contenu de la partition /dev/sda2. Ainsi, tout ce qu'on voit ou copie sous /mnt est en réalité lu ou écrit sur la carte SD. Lorsqu'on démonte la partition (sudo umount /mnt), le dossier /mnt redevient vide et les fichiers restent stockés sur la carte SD.

Maintenant que la partition est montée sur le poste de travail, on décompresse le fichier rootfs.tar.gz dans la partition qui est en fait /mnt sur le poste de travail.

Opération de compression dans /mnt :
sudo tar -xvf ~/Téléchargements/rootfs.tar.gz -C /mnt

NB : il faut enfin démonter la carte par : sudo umount /mnt

A ce niveau la carte est prête à être utilisée.

3 -Ecriture du code C à exécuter

```
#include <stdlib.h>
#include <stdio.h>

int main(){
        printf(" Hello word \n " );
        return EXIT_SUCCESS;
        }
```

4- Compilation du code

Le poste de travail sur basé sur le processeur Intel, donc tout compilation donne un fichier exécutable sur un processeur Intel. La compilation classique se fait comme suit :

```
hello_userspace$ g++ -Wall -o hello hello.c
```

Le résultat de l'exécution donne :

```
Hello word
```

Mais on veut exécuter le « hello Word » sur un périphérique basé sur ARM, pour cela on utilise la **cross-compilation ou compilation croisée.** Cette méthode de compilation permet de compiler un fichier exécutable sur ARM (également plus sur Intel).
Pour y arriver il faut configurer l'environnement de travail, en configurant les variables d'environnement.
Dans notre cas on veut faire un cross-compilation Xilinx Arm Linux. On tape les commandes suivantes pour faire les configurations :

```
$ xilinx_vivado22.2_env
$ export CROSS_COMPILE=arm-linux-gnueabihf-
$ export ARCH=arm
```

Pour vérifier si le poste de travail trouve la commande de compilation, il faut taper :

```
$ arm-linux-gnueabihf-gcc -v
```

Résultat :

```
ocal/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petalinux-linux/us
linux-linux/usr/bin/arm-xilinx-linux-gnueabi --sbindir=/usr/local/
m-xilinx-linux-gnueabi --libexecdir=/usr/local/oe-sdk-hardcoded-bu
gnueabi --datadir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x
ded-buildpath/sysroots/x86_64-petalinux-linux/etc --sharedstatedir
x/com --localstatedir=/usr/local/oe-sdk-hardcoded-buildpath/sysroo
ildpath/sysroots/x86_64-petalinux-linux/usr/lib/arm-xilinx-linux-g
6_64-petalinux-linux/usr/include --oldincludedir=/usr/local/oe-sdk
fodir=/usr/local/oe-sdk-hardcoded-buildpath/sysroots/x86_64-petali
th/sysroots/x86_64-petalinux-linux/usr/share/man --disable-silent-
hatle/git/internal/2022/build-toolchain/tmp/work/x86_64-nativesdk-
h-gnu-ld --enable-shared --enable-languages=c,c++ --enable-threads
ong-long --enable-symvers=gnu --enable-libstdcxx-pch --program-pre
l-libiberty --disable-libssp --enable-libitm --enable-lto --disabl
linker-build-id --with-ppl=no --with-cloog=no --enable-checking=re
=/not/exist/usr/include/c++/11.2.0 --with-build-time-tools=/scratc
petalinux-linux/gcc-cross-canadian-arm/11.2.0-r0/recipe-sysroot-na
h-build-sysroot=/scratch/mhatle/git/internal/2022/build-toolchain/
.0-r0/recipe-sysroot --enable-standard-branch-protection --enable-
version=2.28 --enable-initfini-array
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 11.2.0 (GCC)
```

On peut maintenant compiler le code c.

```
hello_userspace$ arm-linux-gnueabihf-gcc -Wall -o  hello  hello.c
```

4- Exécution du code sur Linux embarqué sur la carte Zedboard

D'abord on monte la partition rootfs et on copie le fichier exécutable dans cette partition

```
$ sudo mount /dev/sda2 /mnt
$ sudo cp /home/else4-g1/Zedboard/hello_userspace /mnt

$ sudo cp /home/else4-g1/Zedboard/hello_userspace/hello /mnt
```

NB : Il est prudent de prendre le chemin complet du fichier, et de démonter la partition avant de retirer la carte du l'unité centrale.

Exécution :
Configuration de l'environnement embarqué (sur xminicom)

```
ls -als
cd /home
mkdir elec4
cd elec4
dmesg
dmesg | tail
sudo -s
mount /dev/mmcblk0p2 /media
cd /media
./test
cd ..
umount /media
shutdown -h now
```

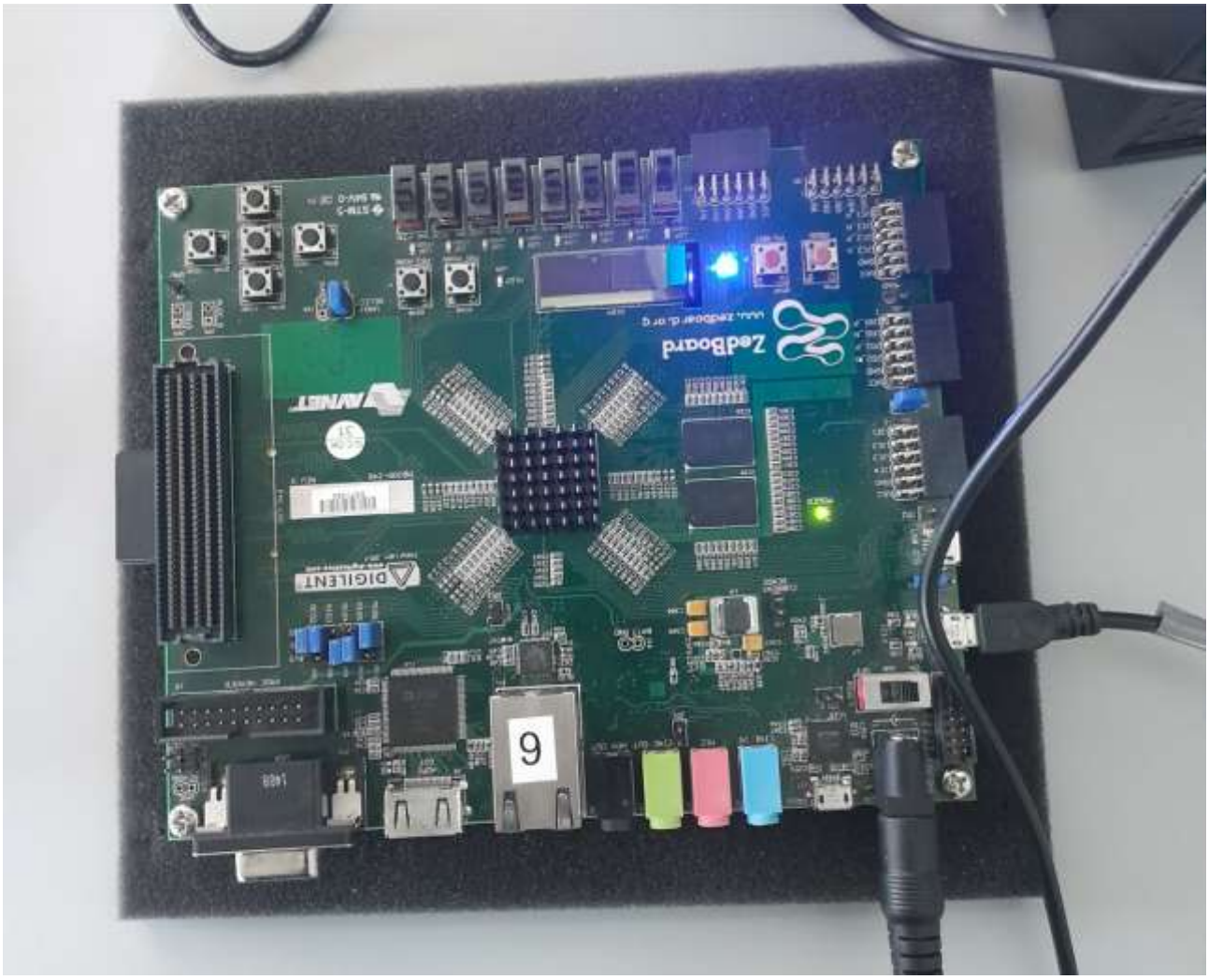
Exécution du code :
On monte le media de la carte SD dans le fichier média du Linux embarqué sur la carte.
Le fichier exécutables se trouve donc dans le media du linux embarqué, et pour l'exécuter on faire comme avec une exécution normale c'est-à-dire **./fichier_executable**

```
zedboard-petalinux:/home$ cd elec4
zedboard-petalinux:/home/elec4$ sudo -s
zedboard-petalinux:/home/elec4# mount /dev/mmcblk0p2 /media
zedboard-petalinux:/home/elec4# cd /media
zedboard-petalinux:/media# ./hello
 Hello word
```

On voit bien que « hello word » est executé.

NB : Pour retirer la carte, il faut d'avoir sortir de Linux avec la commande « shutdown -h now » après cd ~ et attendre à avoir le message « System shalted ».

```
reboot: System halted
```


ZedBoard
DIGILENT
9